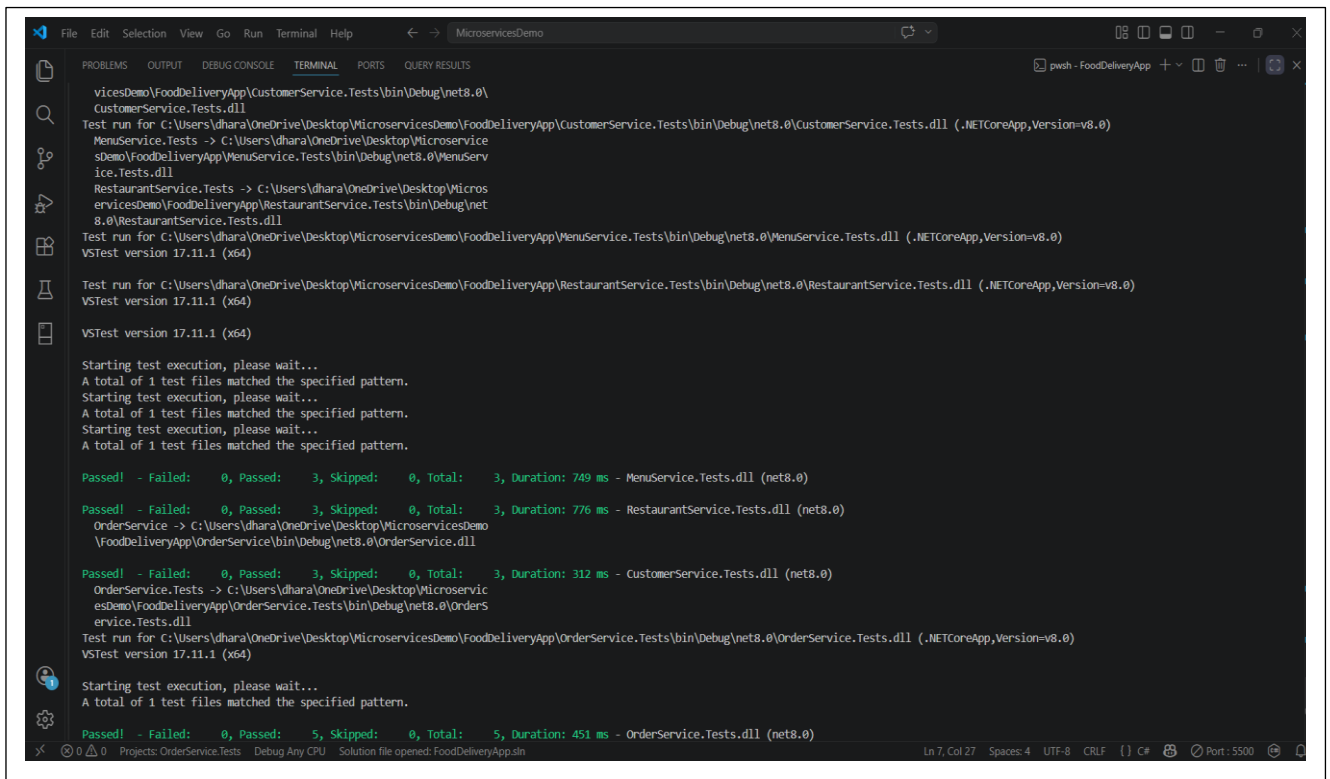


Python Projects:

I added Python projects in WEEK 8 Assignment and Food Delivery Platform Image is Blur So could you please allow to upload in this document



```
MicroservicesDemo
File Edit Selection View Go Run Terminal Help
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
pwsh - FoodDeliveryApp + - - - - -

vicesDemo\FoodDeliveryApp\CustomerService.Tests\bin\Debug\net8.0\
CustomerService.Tests.dll
Test run for C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\CustomerService.Tests\bin\Debug\net8.0\CustomerService.Tests.dll (.NETCoreApp,Version=v8.0)
MenuService.Tests -> C:\Users\dhara\OneDrive\Desktop\Microservice
sDemo\FoodDeliveryApp\MenuService.Tests\bin\Debug\net8.0\MenuServ
ice.Tests.dll
RestaurantService.Tests -> C:\Users\dhara\OneDrive\Desktop\Micros
ervicesDemo\FoodDeliveryApp\RestaurantService.Tests\bin\Debug\net
8.0\RestaurantService.Tests.dll
Test run for C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\MenuService.Tests\bin\Debug\net8.0\MenuService.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Test run for C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\RestaurantService.Tests\bin\Debug\net8.0\RestaurantService.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

VSTest version 17.11.1 (x64)

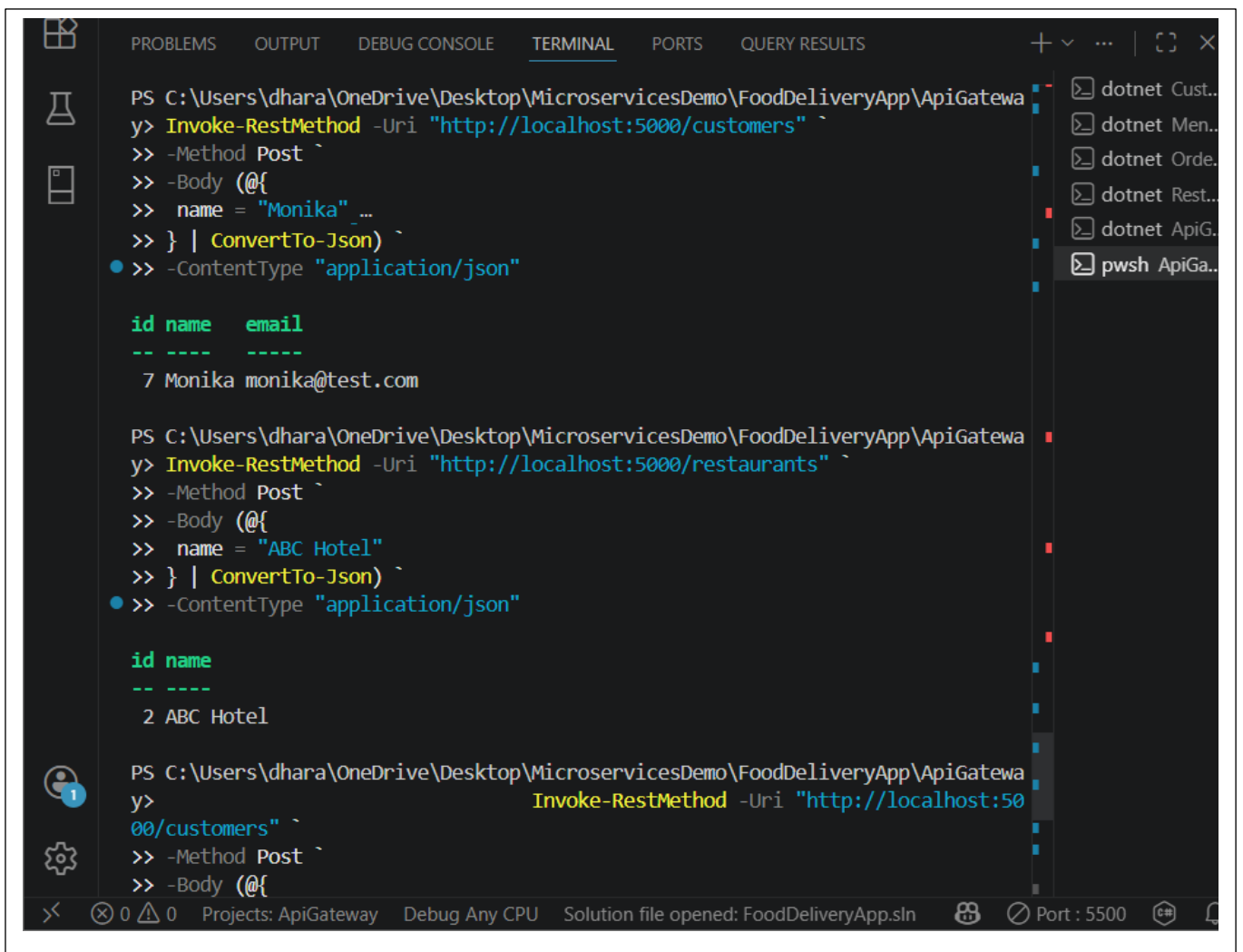
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 3, Skipped: 0, Total: 3, Duration: 749 ms - MenuService.Tests.dll (net8.0)

Passed! - Failed: 0, Passed: 3, Skipped: 0, Total: 3, Duration: 776 ms - RestaurantService.Tests.dll (net8.0)
OrderService -> C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo
\FoodDeliveryApp\OrderService\bin\Debug\net8.0\OrderService.dll
Passed! - Failed: 0, Passed: 3, Skipped: 0, Total: 3, Duration: 312 ms - CustomerService.Tests.dll (net8.0)
OrderService.Tests -> C:\Users\dhara\OneDrive\Desktop\Microservic
esDemo\FoodDeliveryApp\OrderService.Tests\bin\Debug\net8.0\Orders
ervice.Tests.dll
Test run for C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\OrderService.Tests\bin\Debug\net8.0\OrderService.Tests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5, Duration: 451 ms - OrderService.Tests.dll (net8.0)
Ln 7, Col 27 Spaces:4 UTF-8 CRLF {} CF Port: 5500
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGateway>
y> Invoke-RestMethod -Uri "http://localhost:5000/customers" `
>> -Method Post `
>> -Body (@{
>> name = "Monika" ...
>> } | ConvertTo-Json) `
● >> -ContentType "application/json"

id name email
-- --
7 Monika monika@test.com

PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGateway>
y> Invoke-RestMethod -Uri "http://localhost:5000/restaurants" `
>> -Method Post `
>> -Body (@{
>> name = "ABC Hotel"
>> } | ConvertTo-Json) `
● >> -ContentType "application/json"

id name
-- --
2 ABC Hotel

PS C:\Users\dhara\OneDrive\Desktop\MicroservicesDemo\FoodDeliveryApp\ApiGateway>
y> Invoke-RestMethod -Uri "http://localhost:5000/customers" `
>> -Method Post `
>> -Body (@{
dotnet Cust..
dotnet Men..
dotnet Orde..
dotnet Rest..
dotnet ApiG..
pwsh ApiGa..
Port : 5500
```

Agile Scenario-Based Questions:

1. because it embraces changing requirements and allows continuous feedback and iterative development.
2. Waterfall (Traditional SDLC) => because requirements are fixed and it follows a structured, predictable sequence with clear deadlines.
3. Agile (Scrum) => because it delivers working software in short, iterative cycles (Sprints) like every 2 weeks.
4. Individuals and interactions (and face-to-face communication) => the team lacks effective daily collaboration and direct communication.
5. Lack of flexibility (Waterfall approach) => the team failed to respond to change, which is against Agile principles.
6. Daily Scrum => it helps the team sync progress, identify blockers, and stay aligned on daily work.
7. Product Backlog => it contains and prioritizes all tasks/features.
8. Planning Poker => a collaborative estimation using cards
9. Velocity => it measures completed story points per sprint.
10. Burndown Chart => it shows remaining work decreasing over time.
11. No => because working software must meet acceptance criteria and pass testing
12. Waterfall => because delivery happens only at the end.
13. Agile => since the team adapts quickly and releases often
14. Sprint Retrospective => where the team discusses what went wrong.
15. Self-organising teams => they work independently without a manager.
16. Acceptance criteria => to make the story clear.
17. Waterfall => feedback comes late, causing delays.
18. Waterfall => it focuses more on documentation.
19. Velocity => used to predict future work.
20. Simplicity => avoiding unnecessary work/features.
21. Working software => it's not complete or usable yet.
22. Customer collaboration => continuous feedback is needed.
23. Bottleneck => something slowing down progress.
24. Story points (relative sizing) => comparing tasks instead of time.
25. Continuous improvement => the team improves after every sprint.

ITIL Scenario-Based Questions:

1. Incident Management => to restore the service quickly before the meeting.
2. Problem Management => to find and fix the root cause of the recurring issue.
3. Change Management => to plan, approve, and safely implement the upgrade.
4. Service Desk => it acts as the first point of contact.
5. SLA (Service Level Agreement) => defines response and resolution times.
6. Continual Improvement => to analyze and improve performance.
7. IT Asset Management => to track devices and ownership.
8. Release Management => to safely move updates to production.
9. Event Monitoring => detecting issues before users report them
10. CEO's laptop issue => higher priority due to high impact and urgency.
11. Log & Categorize => identifying and routing the ticket
12. Investigate & Diagnose => checking systems to find the cause.
13. Close & Review => after confirming the issue is fixed.
14. Start where you are => improve existing tools first
15. Keep it simple and practical => reduce unnecessary steps.
16. Plan => setting goals, budget, and future strategy.
17. Engage => interacting with stakeholders and gathering feedback.
18. Design & Transition => preparing and launching a new service.
19. First Contact Resolution => solved in the first call.
20. P1 (Critical) => highest priority due to major business impact.
21. Change Management => to ensure proper approval and control of changes.

22. IT Asset Management => checking device lifecycle, warranty, and replacement.
23. Efficiency (and self-service improvement) => reducing manual workload and faster resolution.
24. Continual Improvement => regularly reviewing and improving services.
25. Release Management => deploying the application safely with rollback plans

System Maintenance & Operations Scenario-Based Questions:

1. Check which process is using high CPU. Look for abnormal apps or background tasks.
2. Patches were not tested before deployment. No proper staging or validation done.
3. Check logs for Friday night events. Look for scheduled tasks or jobs causing crash.
4. Regular cleanups and disk monitoring missing. No alerts for low storage space.
5. Follow 3-2-1 backup strategy. Keep offsite and tested backups.
6. Software consistency is missing. Versions are not aligned across systems.
7. Backup data immediately. Replace disk before failure.
8. Check network usage and bandwidth during meetings.
9. It needs adaptive maintenance.
10. Find the root cause instead of just restarting.
11. System may slow or crash, so reduce load or upgrade memory.
12. Always test backups before relying on them.
13. It may be a hacking or repeated login attempt.
14. Patch should have been tested before installing.
15. Set up regular cleanup and automate removing temp files.
16. Alerts help detect issues early before failure happens.
17. Check logs and monitor system to catch the issue when it occurs
18. Use rolling updates or schedule during low-usage time.
19. Preventive maintenance.
20. Check antivirus settings and logs for conflict.
21. Perfective maintenance.
22. Adaptive maintenance.
23. 3-2-1 backup rule was ignored.
24. Check RAM, CPU, and browser usage.
25. Logs show what happened and when, e.g., error after a service stops helps find cause